

CSC 4722: Distributed Systems

Course Project: Peer-to-Peer Distributed Chat & Scoreboard System

Course: CSC 4722 – Distributed Systems

Issued: Wednesday, 9th September 2026

Submission Deadline: Wednesday, 16th September 202

Group Size: Up to 10 Students per Group

1. Overview & Objectives

In this practical lab, your group will build a lightweight, peer-to-peer (P2P) **Distributed Chat and Shared Scoreboard System** using **Java 11+** without external frameworks. Each team member will run an instance (node) on a distinct port (e.g., 8000 through 8009).

By completing this assignment, your team will practically demonstrate four core distributed systems concepts:

- i. **Logical Clocks:** Causal and total ordering of messages across nodes using **Lamport** and **Vector Timestamps**.
- ii. **Distributed Mutual Exclusion:** Coordinating access to a shared resource (the high-score table) using a **Token Ring** mechanism.
- iii. **Leader Election:** Dynamically electing a primary Room Host using the **Bully Election Algorithm**.
- iv. **Resilience & Web Services:** Exposing lightweight REST HTTP endpoints to communicate, pass state, and gracefully recover when nodes crash.

2. System Architecture & Skeleton Code

Your implementation must build upon the provided Java HTTP server/client skeleton code (com.sun.net.httpserver.HttpServer and java.net.http.HttpClient).

```
src/
├── Node.java           // Main application runner
├── models/
│   ├── Message.java   // Chat message structure
│   └── Clock.java      // Lamport & Vector Clock logic
├── sync/
│   └── MutualExclusion.java // Token Ring critical section
├── implementation
│   └── Election.java   // Bully algorithm implementation
├── api/
│   ├── ChatHandler.java // REST API endpoints
│   └── NetworkClient.java // Helper for sending HTTP POST/GET
requests
```

2.1 Clock.java

```
package models;

import java.util.Arrays;

public class Clock {
    private int lamportTime = 0;
    private final int[] vectorClock;
    private final int nodeId;

    public Clock(int nodeId, int totalNodes) {
        this.nodeId = nodeId;
        this.vectorClock = new int[totalNodes];
    }

    // Local event tick
    public synchronized void tick() {
        lamportTime++;
        vectorClock[nodeId]++;
    }

    // Update clocks upon receiving a message
    public synchronized void updateOnReceive(int incomingLamport, int[] incomingVector) {
        // TODO: Implement Lamport clock update: max(local, incoming) + 1
        // TODO: Implement Vector clock update: max(local[i], incoming[i]) for all indices
        // TODO: Always increment local node position after merge
    }

    public synchronized int getLamportTime() { return lamportTime; }
    public synchronized int[] getVectorClock() { return Arrays.copyOf(vectorClock,
vectorClock.length); }
}
```

2.2 MutualExclusion.java

```
package sync;

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class MutualExclusion {
    private final int nodeId;
    private final int nextPeerPort;
    private boolean wantsToUpdateScore = false;
    private boolean hasToken = false;
    private final HttpClient client = HttpClient.newHttpClient();
}
```

```

public MutualExclusion(int nodeId, int nextPeerPort, boolean startsWithToken) {
    this.nodeId = nodeId;
    this.nextPeerPort = nextPeerPort;
    this.hasToken = startsWithToken;
}

public synchronized void requestCriticalSection() {
    this.wantsToUpdateScore = true;
}

public synchronized void receiveToken() {
    this.hasToken = true;
    if (wantsToUpdateScore) {
        // TODO: Execute Critical Section (Update Shared Scoreboard)
        wantsToUpdateScore = false;
    }
    passToken();
}

private void passToken() {
    // TODO: Send POST request to nextPeerPort carrying token payload
    // Set hasToken = false after successful transmission
}
}

```

2.3 Election.java

```

package sync;

import java.util.List;

public class Election {
    private final int nodeId;
    private final List<Integer> peerPorts;
    private int currentLeaderId;
    private boolean isElectionInProgress = false;

    public Election(int nodeId, List<Integer> peerPorts) {
        this.nodeId = nodeId;
        this.peerPorts = peerPorts;
        this.currentLeaderId = peerPorts.size() - 1; // Highest ID is initial host
    }

    public void startElection() {
        System.out.println("Node " + nodeId + " starting election...");
        isElectionInProgress = true;
        // TODO: Send ELECTION message to all peers with higher node IDs
        // TODO: If no higher node responds within timeout, declare self as Leader
        // TODO: Broadcast COORDINATOR message to all peers if self wins
    }
}

```

```

public void handleElectionMessage(int senderId) {
    // TODO: Send OK reply to sender
    // TODO: If not already in election, start local election run
}

public void handleCoordinatorMessage(int newLeaderId) {
    this.currentLeaderId = newLeaderId;
    this.isElectionInProgress = false;
    System.out.println("New Leader recognized: Node " + newLeaderId);
}
}

```

2.4 ChatHandler.java

```

package api;

import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import models.Clock;
import sync.Election;
import sync.MutualExclusion;

import java.io.IOException;
import java.io.OutputStream;

public class ChatHandler implements HttpHandler {
    private final Clock clock;
    private final MutualExclusion mutex;
    private final Election election;

    public ChatHandler(Clock clock, MutualExclusion mutex, Election election) {
        this.clock = clock;
        this.mutex = mutex;
        this.election = election;
    }

    @Override
    public void handle(HttpExchange exchange) throws IOException {
        String path = exchange.getRequestURI().getPath();
        String method = exchange.getRequestMethod();

        if ("POST".equals(method) && "/api/chat".equals(path)) {
            // TODO: Parse incoming JSON payload (message text, lamport_time, vector_clock)
            // TODO: Invoke clock.updateOnReceive(...)
            // TODO: Store message in local log sorted by time
            sendResponse(exchange, 200, "{\"status\":\"Message Received\"}");
        } else if ("POST".equals(method) && "/api/token".equals(path)) {
            mutex.receiveToken();
            sendResponse(exchange, 200, "{\"status\":\"Token Handled\"}");
        }
    }
}

```

```

    } else if ("POST".equals(method) && "/api/election".equals(path)) {
        // TODO: Extract sender ID and invoke election.handleElectionMessage(...)
        sendResponse(exchange, 200, "{\"status\":\"OK\"}");
    } else if ("GET".equals(method) && "/api/health".equals(path)) {
        sendResponse(exchange, 200, "{\"status\":\"ALIVE\"}");
    } else {
        sendResponse(exchange, 404, "Not Found");
    }
}

private void sendResponse(HttpExchange exchange, int statusCode, String response)
throws IOException {
    exchange.sendResponseHeaders(statusCode, response.getBytes().length);
    OutputStream os = exchange.getResponseBody();
    os.write(response.getBytes());
    os.close();
}
}

```

2.5 Node.java

```

import com.sun.net.httpserver.HttpServer;
import api.ChatHandler;
import models.Clock;
import sync.Election;
import sync.MutualExclusion;

import java.net.InetSocketAddress;
import java.util.Arrays;
import java.util.List;

public class Node {
    public static void main(String[] args) throws Exception {
        if (args.length < 2) {
            System.out.println("Usage: java Node <nodeId> <port>");
            System.exit(1);
        }

        int nodeId = Integer.parseInt(args[0]);
        int port = Integer.parseInt(args[1]);

        // Example configuration for 10 peers running locally on ports 8000-8009
        List<Integer> peerPorts = Arrays.asList(8000, 8001, 8002, 8003, 8004, 8005, 8006,
8007, 8008, 8009);
        int totalNodes = peerPorts.size();
        int nextPeerPort = peerPorts.get((nodeId + 1) % totalNodes);

        Clock clock = new Clock(nodeId, totalNodes);
        MutualExclusion mutex = new MutualExclusion(nodeId, nextPeerPort, nodeId == 0);
    }
}

```

```

    Election election = new Election(nodeId, peerPorts);

    HttpServer server = HttpServer.create(new InetSocketAddress(port), 0);
    server.createContext("/api", new ChatHandler(clock, mutex, election));
    server.setExecutor(null);
    server.start();

    System.out.println("Node " + nodeId + " running on port " + port);
}
}

```

3. Technical Requirements & REST API Specification

Your Java nodes must expose and consume the following endpoints:

- **POST /api/chat**
 - **Payload:** {"sender_id": 1, "text": "Hello", "lamport": 4, "vector": [1, 4, 2]}
 - **Action:** Updates local clock state, merges vector clocks, and appends the message to the chronologically sorted log.
- **POST /api/token**
 - **Payload:** {"token_holder": 1, "scores": {...}}
 - **Action:** Passes high-score table access around the logical ring (port = base_port + (id + 1) % N).
- **POST /api/election**
 - **Payload:** {"type": "ELECTION" | "OK" | "COORDINATOR", "sender_id": 2}
 - **Action:** Runs the Bully Election logic when higher node IDs respond or timed-out hosts fail.
- **GET /api/health**
 - **Response:** 200 OK ({"status": "ALIVE"}) used by peers to verify host vitality.

4. Submission Guidelines & Deliverables

Each group needs to present their working solution by **16th September 2026**.

5. Grading Rubric

CSC 4722 PROJECT EVALUATION RUBRIC (TOTAL: 100 MARKS)

i) Logical Clock Implementation (25 Marks)

- Correct Lamport clock increment & message merge. [10 marks]
- Correct Vector clock updates across N nodes. [10 marks]
- Verified consistent message display ordering. [5 marks]

ii) Mutual Exclusion & Token Ring (25 Marks)

- Safe token passing loop between active ports. [10 marks]
- Mutual exclusion maintained (no race conditions in CS). [10 marks]
- Graceful token forward when no CS requested. [5 marks]

iii) Bully Leader Election Algorithm **(25 Marks)**

- Reliable detection of Host node failure via /health. **[5 marks]**
- Correct election message hierarchy and response handling. **[10 marks]**
- Successful coordinator announcement to all active peers. **[10 marks]**

iv) Architecture, Code Quality & API **(15 Marks)**

- Separation of concerns (API, Models, Sync modules). **[10 marks]**
- Robust HTTP error handling & non-blocking execution. **[5 marks]**

|

v) Report & Delegation Documentation **(10 Marks)**

- Accurate UML sequence diagrams. **[5 marks]**
- Clear team task division & test proof logs. **[5 marks]**

6. Plagiarism & Academic Integrity

All submitted code will be screened for similarity. While discussion between groups regarding distributed concepts is encouraged, direct copying of code or using unapproved third-party frameworks will result in a zero mark for all involved parties.